CSS326 Laboratory Assignment #6: Structuring a Web App

Student ID: 6722782164
Project files: 6722782164_lab6.zip (run `npm install`, then `node app.js` → http://localhost:3000)

1. Pool and structure (1.5 pts)

db.js exports a mysql2 pool; book routes live in routes/books.routes.js, mounted at /books in app.js.

Pretty-print


```
[{"id":1,"title":"Clean Code","author":"Robert C. Martin","price":"921.21","stock":8},{"id":2,"title":"The Pragmatic Programmer","author":"Andrew Hunt","price":"610.00","stock":5},{"id":3,"title":"Refactoring","author":"Martin Fowler","price":"700.00","stock":3},{"id":4,"title":"Database Internals","author":"Alex Petrov","price":"850.00","stock":4},{"id":5,"title":"SQL in 10 Minutes","author":"Ben Forta","price":"300.00","stock":12},{"id":6,"title":"Designing Data-Intensive Applications","author":"Martin Kleppmann","price":"950.00","stock":6},{"id":7,"title":"Head First SQL","author":"Lynn Beighley","price":"420.00","stock":9},{"id":8,"title":"Learning MySQL","author":"Seyed Tahaghoghi","price":"640.00","stock":2},{"id":9,"title":"Patterns of Enterprise Application Architecture","author":"Martin Fowler","price":"880.00","stock":3},{"id":10,"title":"MySQL Crash Course","author":"Ben Forta","price":"360.00","stock":10}]
```


GET /books still works

db.js

```
// db.js
// Done for you -- Task 1 asks you to *use* this shared pool throughout,
// not to write it from scratch.
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'bookstore',
  connectionLimit: 10
});

module.exports = pool;
```

app.js

```
// app.js
// This wiring is done for you and should not need to change.
const express = require('express');
const app = express();

app.use(express.json());
app.use(express.static('public'));   // serves public/index.html, style.css, main.js

const booksRouter = require('./routes/books.routes');
app.use('/books', booksRouter);

const PORT = 3000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

routes/books.routes.js

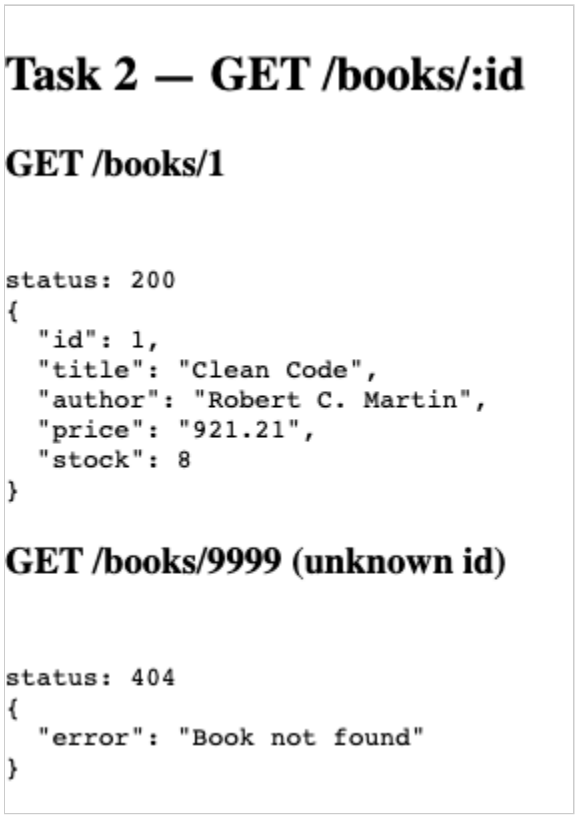
```
// routes/books.routes.js
//
// Thin router: only maps paths to controller functions. Mounted at /books
// in app.js, so '/' here is /books and '/:id' is /books/:id.
const router = require('express').Router();
const c = require('../controllers/books.controller');

router.get('/', c.list);          // GET    /books       (Task 1)
router.get('/:id', c.getOne);     // GET    /books/:id   (Task 2)
router.post('/', c.create);       // POST   /books       (Task 3)
router.put('/:id', c.update);     // PUT    /books/:id   (Task 4)
router.delete('/:id', c.remove);  // DELETE /books/:id   (Task 5)
```

```
module.exports = router;
```

2. Controllers (1.5 pts)

GET /books and GET /books/:id logic is in controllers/books.controller.js (list, getOne). Unknown id → 404.



Task 2 — GET /books/:id

GET /books/1

```
status: 200
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": "921.21",
  "stock": 8
}
```

GET /books/9999 (unknown id)

```
status: 404
{
  "error": "Book not found"
}
```

GET /books/1 → 200; GET /books/9999 → 404

controllers/books.controller.js

```
// controllers/books.controller.js
//
// All the logic and validation for /books lives here; routes/books.routes.js
// only maps HTTP verbs + paths to these functions. books( id, title, author,
// price, stock ). Every value goes through a ? placeholder.
const pool = require('../db');

// GET /books  -- worked example
exports.list = async (req, res) => {
  try {
    const [rows] = await pool.query('SELECT * FROM books');
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

// GET /books/:id                                          Task 2 (1.5 pts)
exports.getOne = async (req, res) => {
  try {
    const [rows] = await pool.query('SELECT * FROM books WHERE id = ?', [
      req.params.id,
    ]);

    if (rows.length === 0) {
      return res.status(404).json({ error: 'Book not found' });
    }

    res.json(rows[0]);
```

```
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

// POST /books                                              Task 3 (1.5 pts)
exports.create = async (req, res) => {
  try {
    const { title, author, price, stock } = req.body;

    if (!title || String(title).trim() === '') {
      return res.status(400).json({ error: 'Title is required' });
    }

    const priceNum = Number(price);
    if (price === undefined || price === '' || !Number.isFinite(priceNum) || priceNum <= 0) {
      return res.status(400).json({ error: 'Price must be a positive number' });
    }

    const [result] = await pool.query(
      'INSERT INTO books (title, author, price, stock) VALUES (?, ?, ?, ?)',
      [String(title).trim(), author || '', priceNum, Number(stock) || 0]
    );

    res.status(201).json({ id: result.insertId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

// PUT /books/:id                                           Task 4 (1.5 pts)
// Updates price and/or stock -- only the fields actually sent in req.body.
exports.update = async (req, res) => {
  try {
    const { price, stock } = req.body;
    const sets = [];
    const values = [];

    if (price !== undefined) {
      const priceNum = Number(price);
      if (!Number.isFinite(priceNum) || priceNum <= 0) {
        return res.status(400).json({ error: 'Price must be a positive number' });
      }
      sets.push('price = ?');
      values.push(priceNum);
    }

    if (stock !== undefined) {
      const stockNum = Number(stock);
      if (!Number.isInteger(stockNum) || stockNum < 0) {
        return res.status(400).json({ error: 'Stock must be a non-negative integer' });
      }
      sets.push('stock = ?');
      values.push(stockNum);
    }

    if (sets.length === 0) {
      return res.status(400).json({ error: 'Provide price and/or stock to update' });
    }

    values.push(req.params.id);
    const [result] = await pool.query(
      `UPDATE books SET ${sets.join(', ')} WHERE id = ?`,
      values
    );

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Book not found' });
    }

    res.json({ updated: result.affectedRows });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

```
// DELETE /books/:id                                        Task 5 (1.5 pts)
exports.remove = async (req, res) => {
  try {
    const [result] = await pool.query('DELETE FROM books WHERE id = ?', [
      req.params.id,
    ]);

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Book not found' });
    }

    res.status(204).end();
  } catch (err) {
    // orders.book_id -> books(id): a book that has orders can't be deleted
    if (err.code === 'ER_ROW_IS_REFERENCED_2') {
      return res
        .status(409)
        .json({ error: 'Cannot delete: this book has existing orders' });
    }
    res.status(500).json({ error: err.message });
  }
};
```

3. Validate POST /books (1.5 pts)

create rejects a missing title or a non-positive price with 400 + message; otherwise inserts with ? placeholders and returns 201 with the new id.

Task 3 — Validate POST /books

POST /books — missing title

```
body: {"author":"Nobody","price":300}
status: 400
{
  "error": "Title is required"
}
```

POST /books — price not positive

```
body: {"title":"Bad Price","price":-10}
status: 400
{
  "error": "Price must be a positive number"
}
```

POST /books — valid

```
body: {"title":"Refactoring Databases","author":"Scott Ambler","price":780,"stock":4}
status: 201
{
  "id": 18
}
```

POST /books: 400 (missing title), 400 (bad price), 201 (valid)

4. Update with PUT (1.5 pts)

PUT /books/:id updates price and/or stock with ? placeholders; 404 if no such id, otherwise reports rows updated.



Task 4 — PUT /books/:id

PUT /books/18 — price and stock

```
body: {"price":820,"stock":6}
status: 200
{
  "updated": 1
}
```

GET /books/18 — after update

```
status: 200
{
  "id": 18,
  "title": "Refactoring Databases",
  "author": "Scott Ambler",
  "price": "820.00",
  "stock": 6
}
```

PUT /books/9999 — unknown id

```
body: {"price":100}
status: 404
{
  "error": "Book not found"
}
```

PUT /books/18 → {updated: 1}; PUT /books/9999 → 404

5. Delete with DELETE (1.5 pts)

DELETE /books/:id removes the book; 404 if not found, otherwise 204.

Task 5 — DELETE /books/:id

DELETE /books/18

```
status: 204
(no content)
```

GET /books/18 — after delete

```
status: 404
{
  "error": "Book not found"
}
```

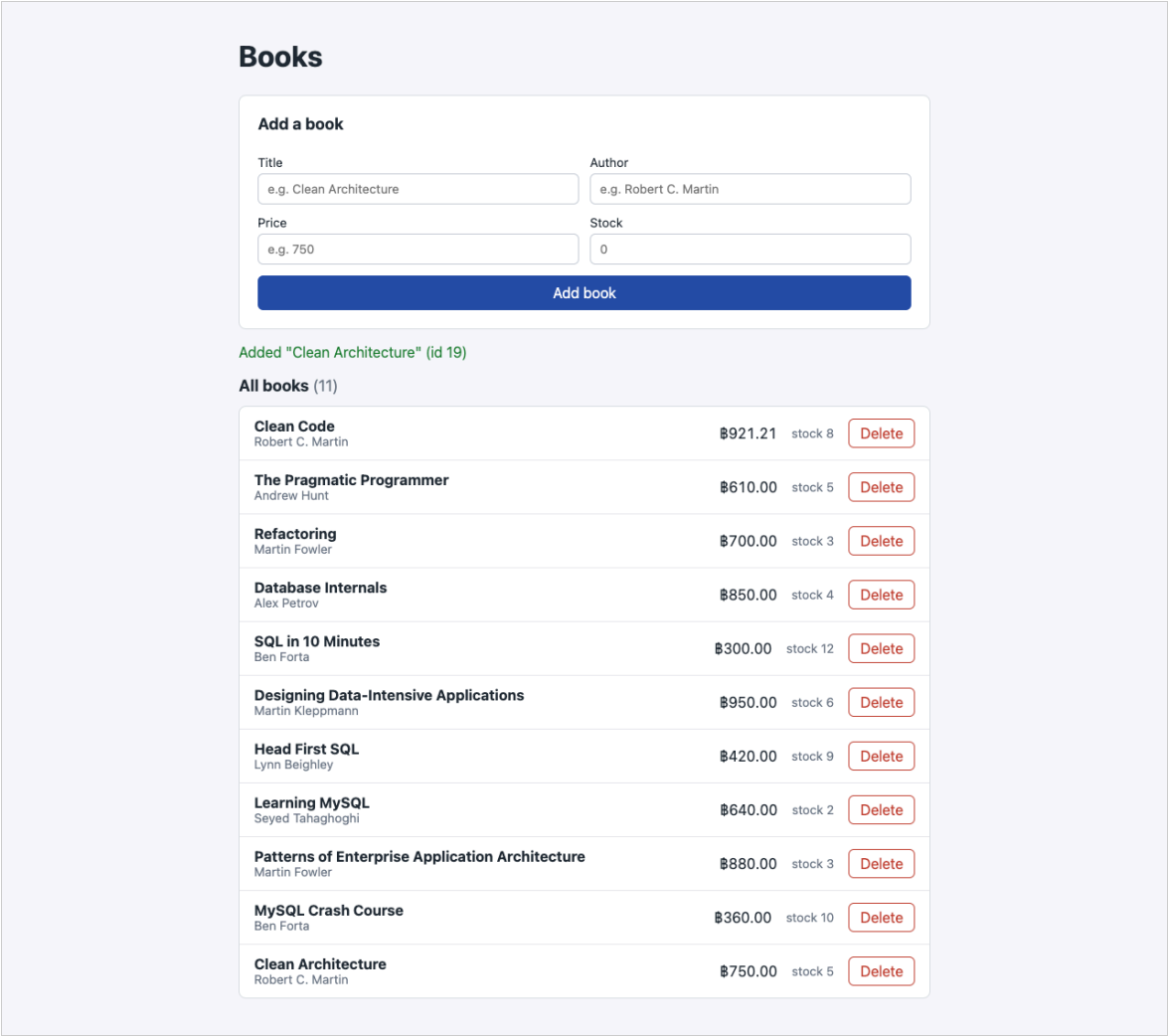
DELETE /books/18 — again (already gone)

```
status: 404
{
  "error": "Book not found"
}
```

DELETE /books/18 → 204; repeat → 404

6. Front-end: list and add (2 pts)

public/index.html, main.js and style.css list all books and add one via a form that POSTs then reloads the list.



Page after adding "Clean Architecture" through the form

`public/index.html`

```
<!-- public/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet" href="style.css">
  <title>Books</title>
</head>
<body>
  <h1>Books</h1>

  <form id="form">
    <h2>Add a book</h2>
    <label>Title <input name="title" placeholder="e.g. Clean Architecture"></label>
    <label>Author <input name="author" placeholder="e.g. Robert C. Martin"></label>
    <label>Price <input name="price" type="number" step="0.01" placeholder="e.g. 750"></label>
    <label>Stock <input name="stock" type="number" min="0" step="1" placeholder="0"></label>
    <button type="submit">Add book</button>
  </form>

  <p id="message"></p>

  <h2>All books <span id="count"></span></h2>
  <ul id="list"></ul>

  <script src="main.js"></script>
</body>
</html>
```

public/main.js

```
// public/main.js
//
// Task 6: list and add. Task 7: delete.

const list = document.getElementById('list');
const form = document.getElementById('form');
const message = document.getElementById('message');
const count = document.getElementById('count');

function showMessage(text, isError) {
  message.textContent = text;
  message.className = isError ? 'error' : 'ok';
}

// Escape text before putting it into innerHTML
function esc(value) {
  const div = document.createElement('div');
  div.textContent = value ?? '';
  return div.innerHTML;
}

// Task 6: fetch every book and render one row per book
async function load() {
  const res = await fetch('/books');
  const books = await res.json();

  count.textContent = `(${books.length})`;
  list.innerHTML = books
    .map(
      (b) => `
      <li>
        <div class="info">
          <strong>${esc(b.title)}</strong>
          <span class="author">${esc(b.author)}</span>
        </div>
        <span class="price">฿${Number(b.price).toFixed(2)}</span>
        <span class="stock">stock ${b.stock}</span>
        <button class="delete" data-id="${b.id}">Delete</button>
      </li>`
    )
    .join('');
}

// Task 6: POST the form as JSON, then reload the list
form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const body = {
    title: form.elements.title.value,
    author: form.elements.author.value,
    price: form.elements.price.value,
    stock: form.elements.stock.value,
  };

  const res = await fetch('/books', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
  });
  const data = await res.json();

  if (!res.ok) {
    showMessage(data.error, true);
    return;
  }

  showMessage(`Added "${body.title}" (id ${data.id})`, false);
  form.reset();
  load();
});

// Task 7: one listener on the list handles every Delete button
list.addEventListener('click', async (e) => {
```

```
  if (!e.target.matches('button.delete')) return;

  const id = e.target.dataset.id;
  const res = await fetch(`/books/${id}`, { method: 'DELETE' });

  if (!res.ok) {
    const data = await res.json();
    showMessage(data.error, true);
    return;
  }

  showMessage(`Deleted book id ${id}`, false);
  load();
});

load();
```

public/style.css

```
/* public/style.css */

body {
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
  max-width: 760px;
  margin: 40px auto;
  padding: 0 16px;
  color: #1f2933;
  background: #f7f8fa;
}

h1 { margin-bottom: 24px; }
h2 { font-size: 1.1rem; margin: 0 0 12px; }

#form {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  padding: 20px;
  background: #fff;
  border: 1px solid #d9dee5;
  border-radius: 8px;
  margin-bottom: 16px;
}
#form h2, #form button { grid-column: 1 / -1; }

label { display: flex; flex-direction: column; font-size: 0.85rem; gap: 4px; }

input {
  padding: 8px 10px;
  border: 1px solid #c4cbd4;
  border-radius: 6px;
  font: inherit;
}

button {
  background: #2f5fd0;
  color: #fff;
  border: none;
  border-radius: 6px;
  padding: 10px 16px;
  font: inherit;
  cursor: pointer;
}
button:hover { background: #244ba8; }

button.delete {
  background: #fff;
  color: #c0392b;
  border: 1px solid #c0392b;
  padding: 6px 12px;
}
button.delete:hover { background: #c0392b; color: #fff; }

#message { min-height: 1.2em; margin: 0 0 16px; }
```

```
#message.ok { color: #1e7e34; }
#message.error { color: #c0392b; }

#count { color: #6b7685; font-weight: normal; }

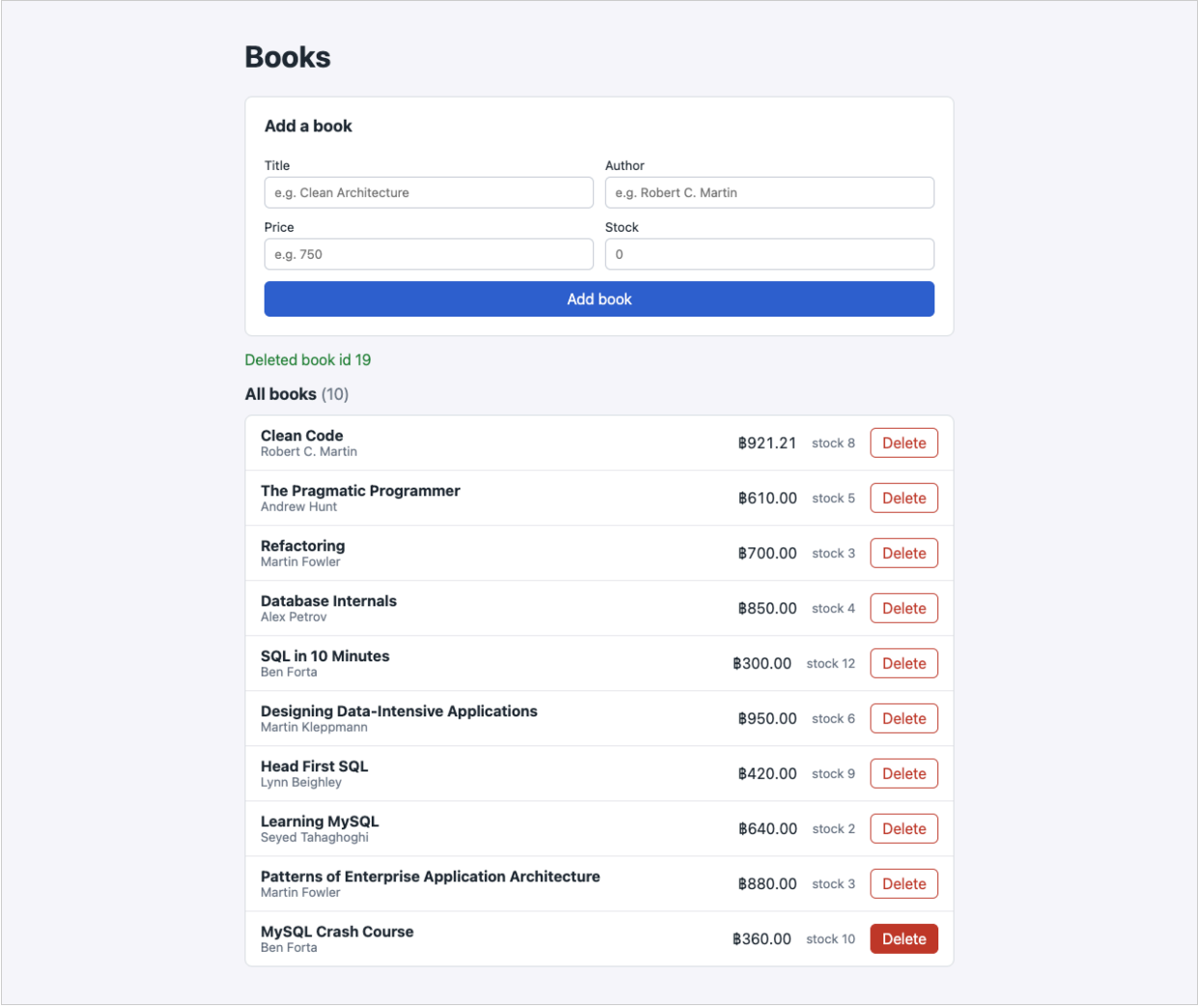
#list {
  list-style: none;
  padding: 0;
  margin: 0;
  background: #fff;
  border: 1px solid #d9dee5;
  border-radius: 8px;
}

li {
  display: flex;
  align-items: center;
  gap: 16px;
  padding: 12px 16px;
  border-bottom: 1px solid #e6e9ee;
}
li:last-child { border-bottom: none; }

.info { flex: 1; display: flex; flex-direction: column; }
.author, .stock { color: #6b7685; font-size: 0.85rem; }
.price { font-variant-numeric: tabular-nums; }
```

7. Front-end: delete (1.5 pts)

Each row has a Delete button that sends DELETE /books/:id and reloads the list (handled in public/main.js above).



"Clean Architecture" removed — list back to 10 books

8. Short answer (1 pt)

A connection pool keeps a set of already-open MySQL connections and reuses them across requests, so the server avoids paying the cost of opening a new connection for every query and can serve several requests at once while capping the total number of connections (connectionLimit) the database has to handle. Putting controllers in their own files separates "what URL maps to what" (the router) from "what actually happens" (the query and validation logic), which keeps each file small and focused and makes the logic easier to find, test and change without touching the routing or app.js.